Qwen-3本地部署与调用详解（上）

Qwen-3本地部署与调用详解（上）
- 一、Qwen3系列模型介绍
  - 1. Qwen3 模型简介
  - 2. Qwen3 模型特性
  - 3. Qwen3 模型训练流程
  - 4.Qwen3模型相关资料
  - 5. Qwen3全系列模型性能及硬件需求介绍
    - 5.1 不同模型适用场景参考
    - 5.2 各模型性能评分及配置参考
  - 6. 公开课课件领取
- 二、Qwen3模型下载与Transformers推理流程
  - 1. Qwen3模型下载与环境搭建
  - 2. Jupyter中使用transformer原生库调用流程
- 三、Qwen3-14B模型接入Ollama与推理流程
  - 1. Ollama项目安装
    - 【安装方案一】Ollama在线安装
    - 【安装方案二】Ollama离线安装
  - 2. Qwen3-14B GGUF格式模型权重下载
    - 2.1 【方案一】在线下载模型权重
    - 2.2 【方案二】离线下载模型权重

一、Qwen3系列模型介绍

1. Qwen3 模型简介

2025年4月29日凌晨，阿里巴巴开源新一代通义千问模型Qwen3，其中旗舰模型 Qwen3-235B-A22B 在代码、数学、通用能力等基准测试中，与 DeepSeek-R1、o1、o3-mini、Grok-3 和 Gemini-2.5-Pro 等顶级模型相比，表现出极具竞争力的结果。此外，小型 MoE 模型 Qwen3-30B-A3B 的激活参数数量是 QwQ-32B 的 10%，表现更胜一筹，甚至像 Qwen3-4B 这样的小模型也能匹敌 Qwen2.5-72B-Instruct 的性能，性能全面超越Deepseek-R1、豆包1.5pro 等国内领先模型，并登顶全球最强开源模型。

	Qwen3-235B-A22B MoE	Qwen3-32B Dense	OpenAI-o1 2024-12-17	Deepseek-R1	Grok 3 Beta Think	Gemini2.5-Pro	OpenAI-o3-mini Medium
ArenaHard	95.6	93.8	92.1	93.2	-	96.4	89.0
AIME'24	85.7	81.4	74.3	79.8	83.9	92.0	79.6
AIME'25	81.5	72.9	79.2	70.0	77.3	86.7	74.8
LiveCodeBench v5, 2024.10-2025.02	70.7	65.7	63.9	64.3	70.6	70.4	66.3
CodeForces Elo Rating	2056	1977	1891	2029	-	2001	2036
Aider Pass@2	61.8	50.2	61.7	56.9	53.3	72.9	53.8
LiveBench 2024-11-25	77.1	74.9	75.7	71.6	-	82.4	70.0
BFCL v3	70.8	70.3	67.8	56.9	-	62.9	64.6
MultiIF 8 Languages	71.9	73.0	48.8	67.7	-	77.8	48.4

	Qwen3-30B-A3B MoE	QwQ-32B	Qwen3-4B Dense	Qwen2.5-72B-Instruct	Gemma3-27B-IT	DeepSeek-V3	GPT-4o 2024-11-20
ArenaHard	91.0	89.5	76.6	81.2	86.8	85.5	85.3
AIME'24	80.4	79.5	73.8	18.9	32.6	39.2	11.1
AIME'25	70.9	69.5	65.6	15.0	24.0	28.8	7.6
LiveCodeBench v5, 2024.10-2025.02	62.6	62.7	54.2	30.7	26.9	33.1	32.7
CodeForces Elo Rating	1974	1982	1671	859	1063	1134	864
GPQA	65.8	65.6	55.9	49.0	42.4	59.1	46.0
LiveBench 2024-11-25	74.3	72.0	63.6	51.4	49.2	60.5	52.2
BFCL v3	69.1	66.4	65.9	63.4	59.1	57.6	72.5
MultiIF 8 Languages	72.2	68.3	66.3	65.3	69.8	55.6	65.6

1. AIME 24/25: We sample 64 times for each query and report the average of the accuracy. AIME25 consists of Part I and Part II, with a total of 30 questions.
2. Aider: We didn't activate the think mode of Qwen3 to balance efficiency and effectiveness.
3. BFCL: The Qwen3 models are evaluated using the FC format, while the baseline models are assessed using the highest scores obtained from either the FC or prompt formats.

Qwen3原生支持MCP协议，并具备强大的工具调用（function calling）能力，并且集成封装了工具调用模板和工具调用解析器的Qwen-Agent 框架，大大降低编码复杂性，Qwen3 的工具调用能力，支持实现高效的手机及电脑Agent操作等任务。

2. Qwen3 模型特性

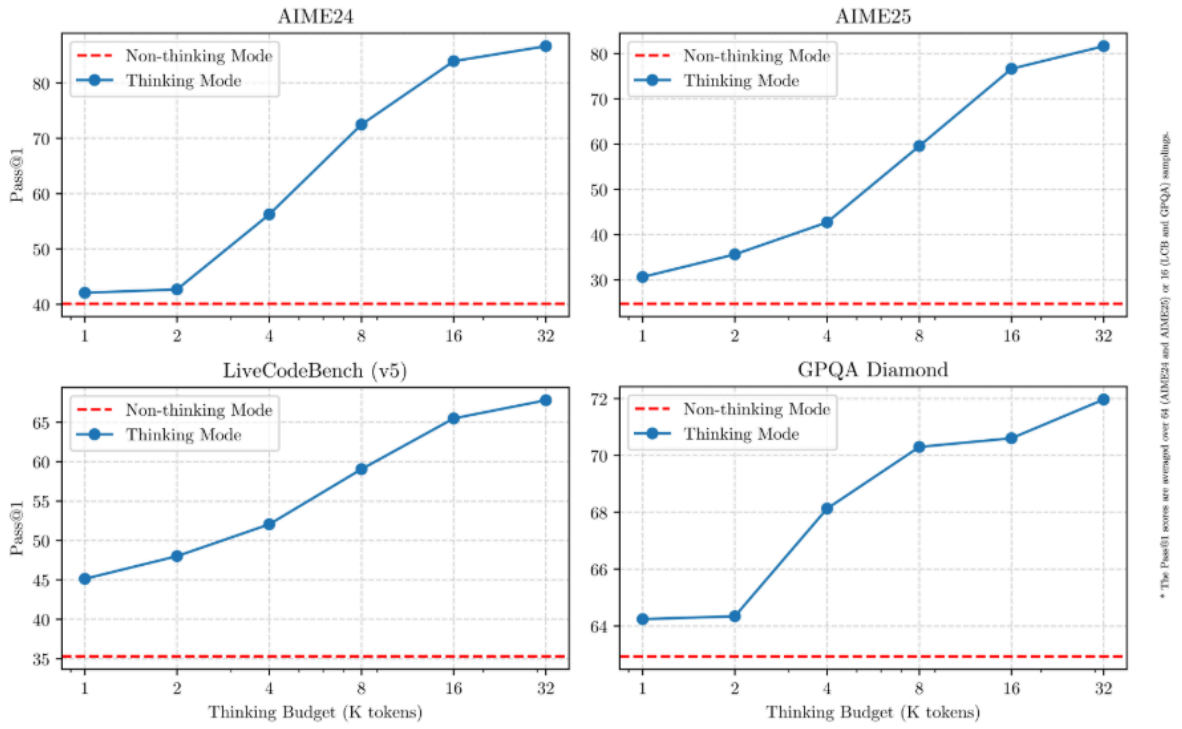
最新版本Qwen3有以下特点：

- 全尺寸稠密与混合专家模型：0.6B, 1.7B, 4B, 8B, 14B, 32B and 30B-A3B, 235B-A22B
- 支持在**思考模式**（用于复杂逻辑推理、数学和编码）和 非思考模式 （用于高效通用对话）之间**无缝切换**，确保在各种场景下的最佳性能。
- 显著增强的推理能力，在数学、代码生成和常识逻辑推理方面超越了之前的 QwQ（在思考模式下）和 Qwen2.5 指令模型（在非思考模式下）。
- 卓越的人类偏好对齐，在创意写作、角色扮演、多轮对话和指令跟随方面表现出色，提供更自然、更吸引人和更具沉浸感的对话体验。
- 擅长智能体能力，可以在思考和非思考模式下精确集成外部工具，在复杂的基于代理的任务中在开源模型中表现领先。
- 支持 100 多种语言和方言，具有强大的多语言理解、推理、指令跟随和生成能力。

同时Qwen3系列模型的核心亮点具备多种思考模式：

- 思考模式：在这种模式下，模型会逐步推理，经过深思熟虑后给出最终答案。这种方法非常适合需要深入思考的复杂问题。
- 非思考模式：在此模式中，模型提供快速、近乎即时的响应，适用于那些对速度要求高于深度的简单问题。

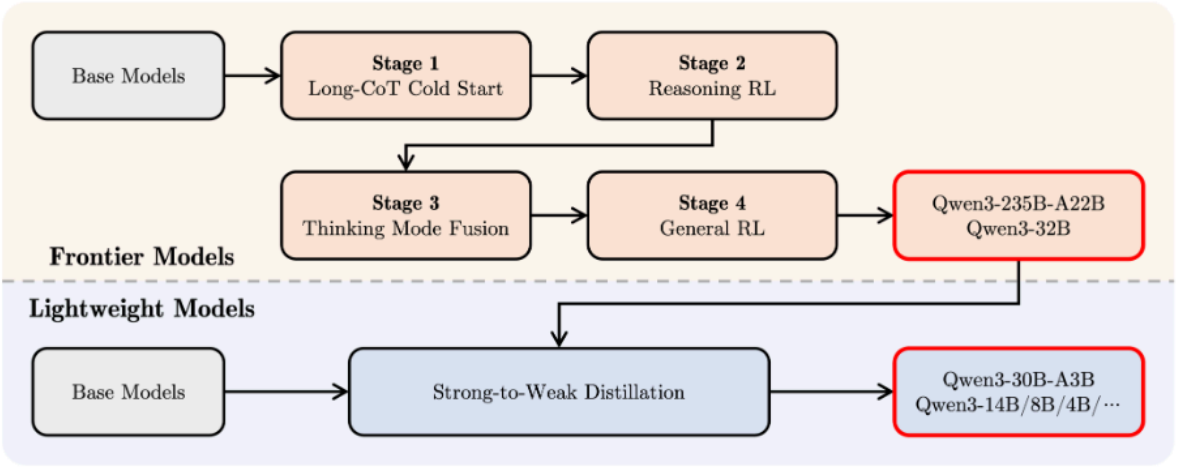
这种灵活性使用户能够根据具体任务控制模型进行“思考”的程度。例如，复杂的问题可以通过扩展推理步骤来解决，而简单的问题则可以直接快速作答，无需延迟。至关重要的是，这两种模式的结合大大增强了模型实现稳定且高效的“思考预算”控制能力。如上文所述，Qwen3 展现出可扩展且平滑的性能提升，这与分配的计算推理预算直接相关。这样的设计让用户能够更轻松地为不同任务配置特定的预算，在成本效益和推理质量之间实现更优的平衡。



3. Qwen3 模型训练流程

在预训练方面，Qwen3 的数据集相比 Qwen2.5 有了显著扩展。Qwen2.5是在 18 万亿个 token 上进行预训练的，而 Qwen3 使用的数据量几乎是其两倍，达到了约 36 万亿个 token，涵盖了 119 种语言和方言。为了构建这个庞大的数据集，我们不仅从网络上收集数据，还从 PDF 文档中提取信息。我们使用 Qwen2.5-VL 从这些文档中提取文本，并用 Qwen2.5 改进提取内容的质量。为了增加数学和代码数据的数量，我们利用 Qwen2.5-Math 和 Qwen2.5-Coder 这两个数学和代码领域的专家模型合成数据，合成了包括教科书、问答对以及代码片段等多种形式的数据。

预训练过程分为三个阶段。在第一阶段（S1），模型在超过 30 万亿个 token 上进行了预训练，上下文长度为 4K token。这一阶段为模型提供了基本的语言技能和通用知识。在第二阶段（S2），我们通过增加知识密集型数据（如 STEM、编程和推理任务）的比例来改进数据集，随后模型又在额外的 5 万亿个 token 上进行了预训练。在最后阶段，我们使用高质量的长上下文数据将上下文长度扩展到 32K token，确保模型能够有效地处理更长的输入。



为了开发能够同时具备思考推理和快速响应能力的混合模型，Qwen3模型实施了一个四阶段的训练流程。该流程包括：（1）长思维链冷启动，（2）长思维链强化学习，（3）思维模式融合，以及（4）通用强化学习。

在第一阶段，使用多样的的长思维链数据对模型进行了微调，涵盖了数学、代码、逻辑推理和 STEM 问题等多种任务和领域。这一过程旨在为模型配备基本的推理能力。第二阶段的重点是大规模强化学习，利用基于规则的奖励来增强模型的探索和钻研能力。

在第三阶段，在一份包括长思维链数据和常用的指令微调数据的组合数据上对模型进行微调，将非思考模式整合到思考模型中。确保了推理和快速响应能力的无缝结合。最后，在第四阶段，在包括指令遵循、格式遵循和 Agent 能力等在内的 20 多个通用领域的任务上应用了强化学习，以进一步增强模型的通用能力并纠正不良行为。

4.Qwen3模型相关资料

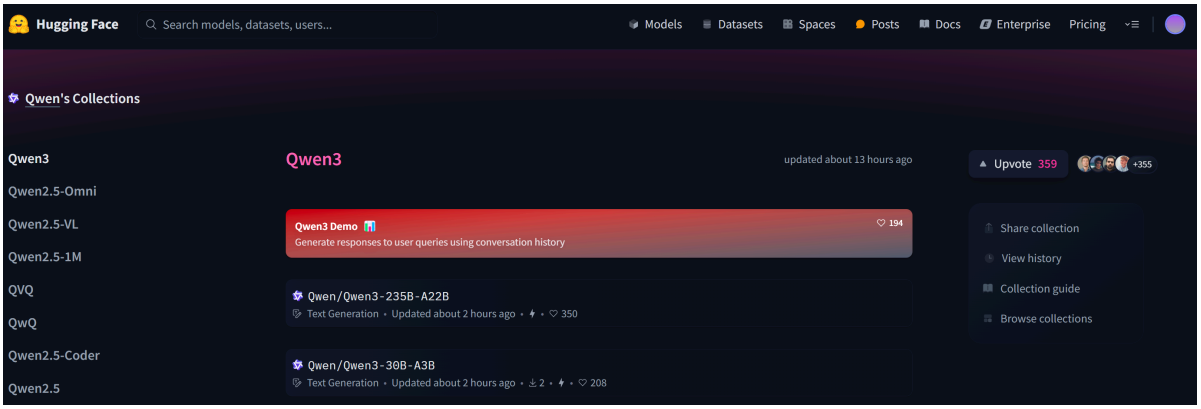
- Qwen3模型技术博客：https://qwenlm.github.io/zh/blog/qwen3/



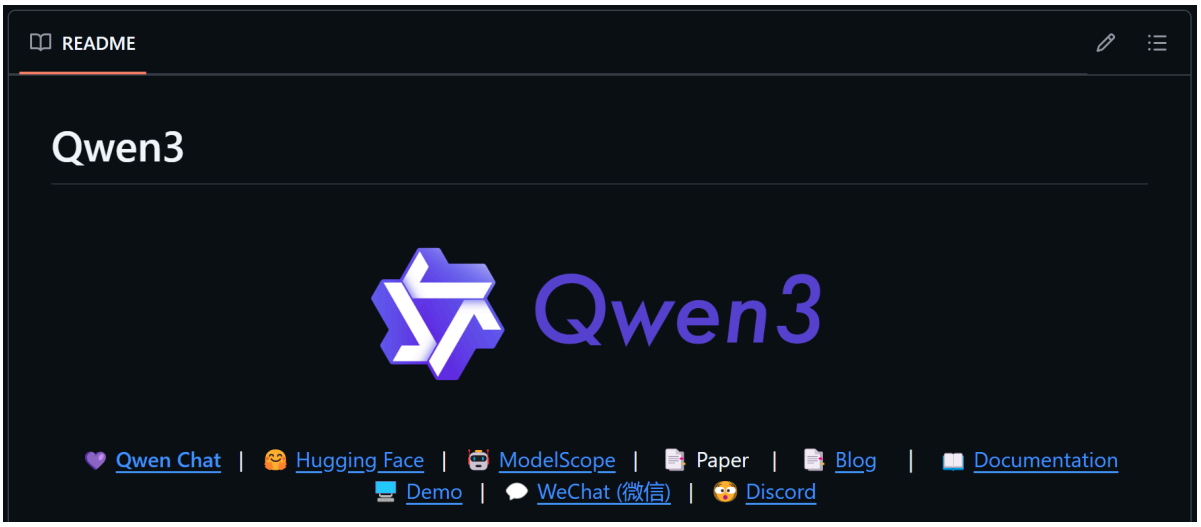
- Qwen3系列魔搭社区主页：https://modelscope.cn/collections/Qwen3-9743180bdc6b48



- Qwen3系列 huggingface：https://huggingface.co/collections/Qwen/qwen3-67dd247413f0e2e4f653967f



- Qwen3模型GitHub主页：https://github.com/QwenLM/Qwen3



- Qwen3模型入门介绍：https://www.bilibili.com/video/BV1fGGyzKEJf/



5. Qwen3全系列模型性能及硬件需求介绍

5.1 不同模型适用场景参考


8款不同尺寸
Dense 模型
Qwen3-0.6B
Qwen3-1.7B
Qwen3-4B
Qwen3-8B
Qwen3-14B
Qwen3-32B
MoE 模型
Qwen3-30B-A3B
Qwen3-235B-A22B
如何选择适合的模型？
本地测试及科研
Qwen3-0.6B/1.7B，硬件要求低，适合快速实验
手机端侧应用
Qwen3-4B，性能与效率兼顾，适合移动端部署
电脑或汽车端
Qwen3-8B，适用于对话系统、语音助手等场景
企业落地
Qwen3-14B/32B 性能更强，适合复杂任务
云端高效部署
MoE 模型，Qwen3-30B-A3B 速度快
Qwen3-235B-A22B 性能强劲且显存占用低
无论你是开发者、科研人员还是普通用户
Qwen3 都能满足你的需求！

5.2 各模型性能评分及配置参考



> 注：Qwen3-235B模型最低也可以在H20（98G）*4服务器上运行，但这种配置下运行非常极限，更推荐在H20（141G）*4服务器上运行，以获得更好的性能。

6. 公开课课件领取

- **课件领取地址**



- 公开课服务器配置【AutoDL服务器租赁运行】
  - PyTorch 2.5.1，Python 3.12(ubuntu22.04)，Cuda 12.4
  - 操作系统：Ubuntu 22.04
  - **GPU**：H800 * 4
  - **CPU**：40 vCPU Intel(R) Xeon(R) Platinum 8458P
  - **内存**：200GB DDR4

  基于此，公开课中将演示30B-A3B模型和8B dense模型部署和运行方法，此外大家也可以根据自己实际硬件情况合理选择模型进行部署和运行。

二、Qwen3模型下载与Transformers推理流程

1. Qwen3模型下载与环境搭建

- 【可选】创建虚拟环境

```
conda create --name qwen3 python=3.11
conda init
source ~/.bashrc
conda activate qwen3
```

```
root@autodl-container-8f1a459ec2-4b1a8fdf:~/autodl-tmp# conda init
no change     /root/miniconda3/condabin/conda
no change     /root/miniconda3/bin/conda
no change     /root/miniconda3/bin/conda-env
no change     /root/miniconda3/bin/activate
no change     /root/miniconda3/bin/deactivate
no change     /root/miniconda3/etc/profile.d/conda.sh
no change     /root/miniconda3/etc/fish/conf.d/conda.fish
no change     /root/miniconda3/shell/condabin/Conda.psm1
no change     /root/miniconda3/shell/condabin/conda-hook.ps1
no change     /root/miniconda3/lib/python3.12/site-packages/xontrib/conda.xsh
no change     /root/miniconda3/etc/profile.d/conda.csh
modified      /root/.bashrc

==> For changes to take effect, close and re-open your current shell. <==

root@autodl-container-8f1a459ec2-4b1a8fdf:~/autodl-tmp# source ~/.bashrc
```

- 【可选，若创建虚拟环境，则必须要执行】创建Jupyter Kernel

```
conda install jupyterlab
conda install ipykernel
python -m ipykernel install --user --name qwen3 --display-name "Python qwen3"
```

```
Platform: linux-64
Collecting package metadata (repodata.json): done
Solving environment: done

# All requested packages already installed.

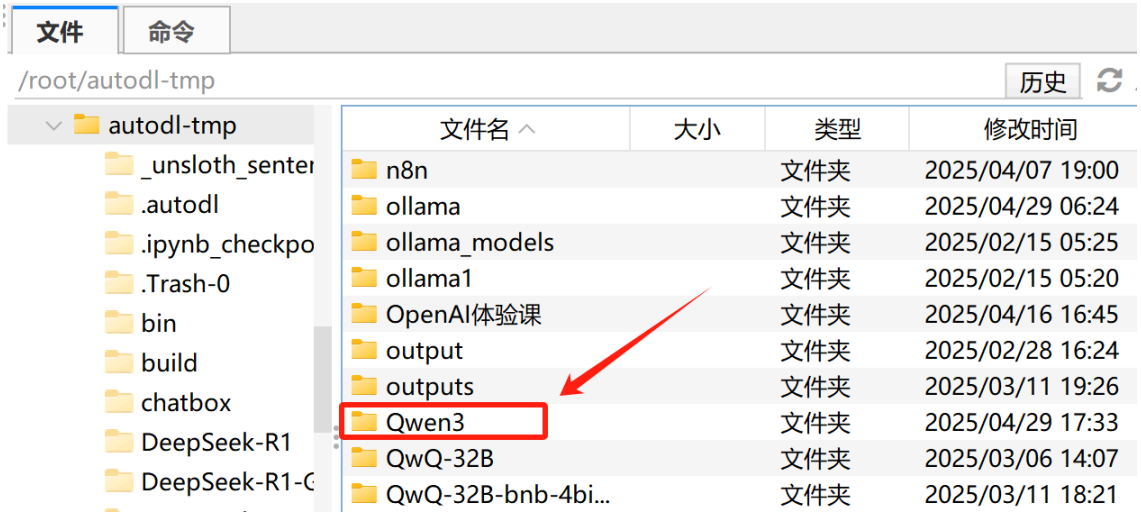
(R1) root@autodl-container-d83e478b47-366a2541:~# python -m ipykernel install --user --name R1 --display-name "Python
R1)"
Installed kernelspec R1 in /root/.local/share/jupyter/kernels/r1
```

- 安装魔搭社区

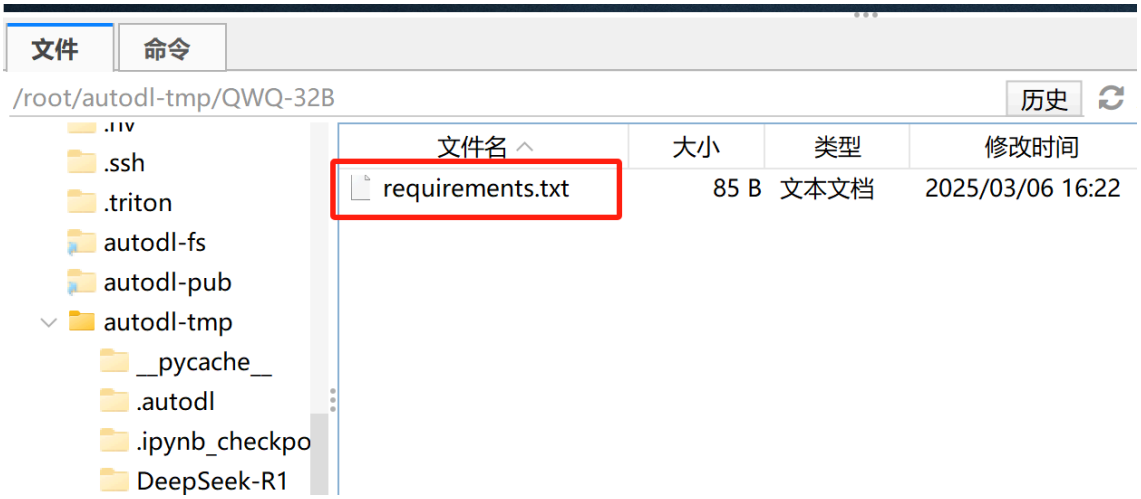
```
pip install modelscope
```

- 创建项目主目录

```
cd /root/autodl-tmp
mkdir Qwen3
```



- 上传并安装依赖



项目依赖在网盘中：



上传完即可进行安装

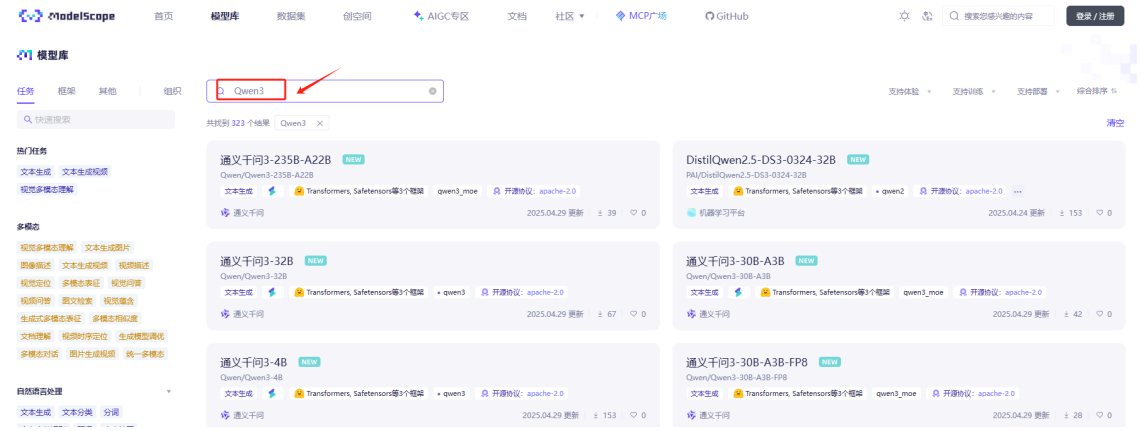
```
pip install -r requirements.txt
```

- 下载项目权重

一般来说我们可以在魔搭社区或者hugging上下载，考虑到国内网络环境，优先推荐在魔搭社区上选择模型版本并进行下载。进入到`ModelScope`的主页中：https://www.modelscope.cn/home



通过`Qwen3`关键词找到对应的模型详情页，点击进入，如下图所示：





这里我们选择14B模型进行下载和测试：

```
cd /root/autodl-tmp/Qwen3
mkdir ./Qwen3-14B
modelscope download --model Qwen/Qwen3-14B --local_dir ./Qwen3-14B
```

```
(base) root@autodl-container-b4de43b42c-6d000661:~# cd ./autodl-tmp/
(base) root@autodl-container-b4de43b42c-6d000661:~/autodl-tmp# mkdir QwQ-32B
(base) root@autodl-container-b4de43b42c-6d000661:~/autodl-tmp# modelscope download --model Qwen/QwQ-32B --local_dir ./
QwQ-32B
Downloading Model to directory: /root/autodl-tmp/QwQ-32B
Downloading [config.json]: 100%|████████████████████████████████████████| 662/662 [00:00<00:00, 2.02kB/s]
Downloading [configuration.json]: 100%|████████████████████████████████| 48.0/48.0 [00:00<00:00, 102B/s]
Downloading [added_tokens.json]: 100%|██████████████████████████████████| 707/707 [00:00<00:00, 1.47kB/s]
Downloading [generation_config.json]: 100%|█████████████████████████████| 243/243 [00:00<00:00, 497B/s]
Downloading [model-00005-of-00014.safetensors]:   0%|                     | 0.00/4.54G [00:00<?, ?B/s]
Downloading [configuration.json]: 100%|████████████████████████████████| 48.0/48.0 [00:00<00:00, 102B/s]
Downloading [generation_config.json]: 100%|█████████████████████████████| 243/243 [00:00<00:00, 497B/s]
Downloading [model-00007-of-00014.safetensors]:   0%|                     | 0.00/4.54G [00:00<?, ?B/s]
Downloading [model-00003-of-00014.safetensors]:   0%|                     | 0.00/4.54G [00:00<?, ?B/s]
```



2. Jupyter中使用transformer原生库调用流程

然后即可使用transformer进行调用，具体调用步骤如下：

- 导入相关库

```
from modelscope import AutoModelForCausalLM, AutoTokenizer
```

- 设置模型下载地址

```
model_name = "./Qwen3-14B"
```

- 实例化预训练模型与分词器

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",  # 指定模型权重数据类型
    device_map="auto"  # 指定模型权重加载设备
)
```

- 创建消息message

```
prompt = "给出一个关于大语言模型的简短介绍。"
messages = [
    {"role": "user", "content": prompt}
]
```

- 词嵌入过程

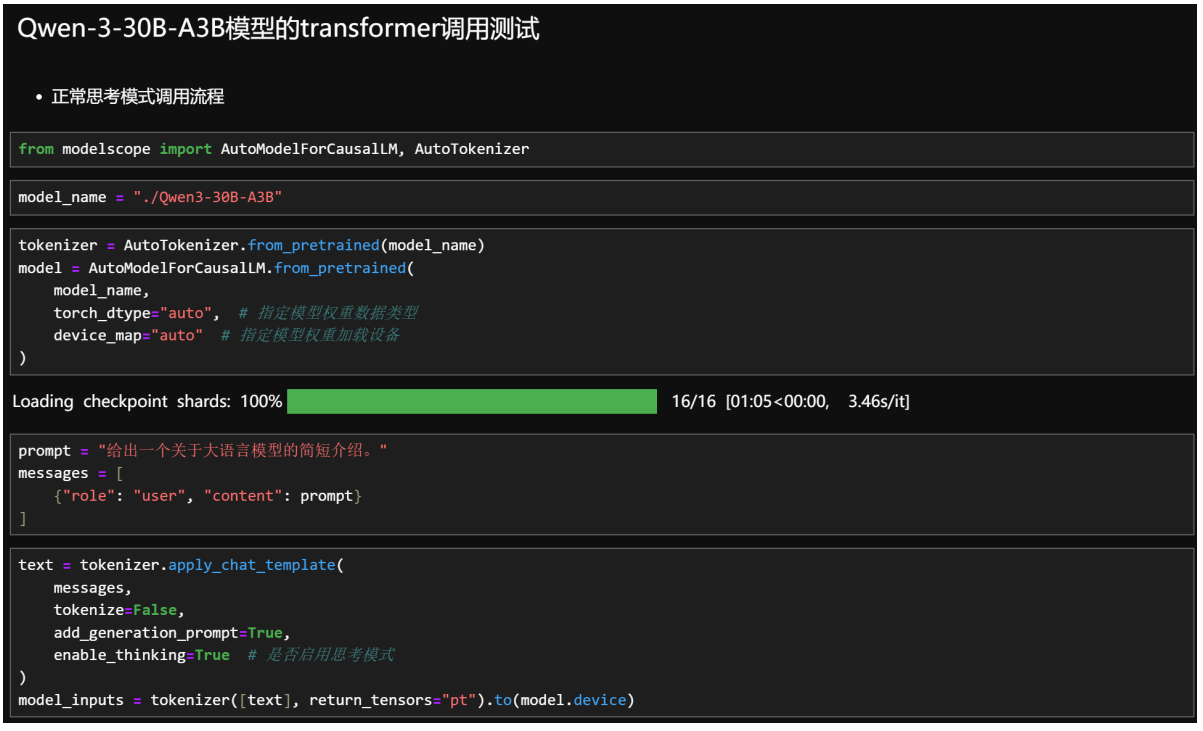
```
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False,
    add_generation_prompt=True,
    enable_thinking=True  # 是否启用思考模式
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)
```

- 创建并回复

```
# 进行文本补全
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=32768
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

# 结果将首先显示思考内容，在<think></think>标签中，然后是实际响应
print(tokenizer.decode(output_ids, skip_special_tokens=True))
```

最终运行结果如下：



```
<think>
嗯，用户让我给出一个关于大语言模型的简短介绍。首先，我需要确定用户的需求是什么。可能他们需要的是一个简洁明了的概述，适合快速了解大语言模型的基本概念。用户可能是学生、研究人员，或者是对AI感兴趣的一般读者。

接下来，我要考虑介绍的结构。通常，简短介绍需要包括定义、核心特点、应用场景以及可能的挑战。但用户要求简短，所以每个部分要尽量精简。可能需要先定义大语言模型，然后提到它们的能力，比如处理多种任务，生成文本，理解上下文等。

然后，我需要确保用词准确且易懂。避免使用太多专业术语，或者如果使用的话，要简单解释。比如，“参数量巨大”这样的术语可能需要解释，但用户可能已经知道，所以可能不需要深入。

还要考虑应用场景，比如自然语言处理、聊天机器人、内容生成等，这些是常见的应用，可以举几个例子。同时，可能需要提到训练数据和训练方法，比如基于大量文本数据的预训练和微调。

另外，用户可能想知道大语言模型的优缺点，比如强大的泛化能力，但可能存在的偏见或计算资源需求。不过，由于是简短介绍，可能不需要深入讨论缺点，除非用户特别指出。

还要注意不要遗漏关键点，比如大语言模型是深度学习模型，基于Transformer架构，这些是重要的背景信息。但可能在简短介绍中，可以简要提到架构，或者只说“基于深度学习技术”更简洁。

最后，检查是否符合字数要求，确保信息全面但不过于冗长。可能需要分点或用段落结构，但用户没有特别要求格式，所以保持连贯的段落即可。
</think>

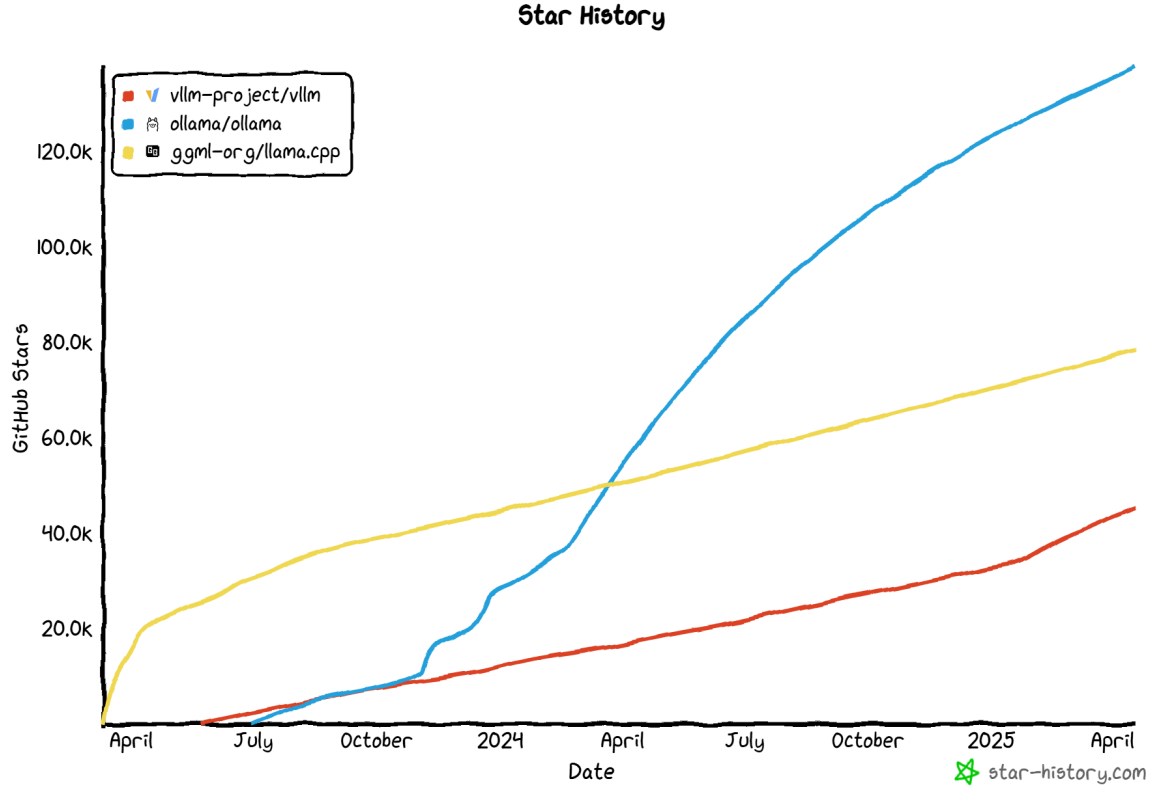
大语言模型（Large Language Models, LLMs）是基于深度学习技术构建的超大规模神经网络模型，通过海量文本数据训练，具备强大的自然语言理解和生成能力。它们能够执行文本创作、问答、翻译、代码生成等多种任务，支持多语言交互，并能根据上下文进行逻辑推理和创造性输出。当前主流模型如GPT、PaLM、通义千问等，依赖于Transformer架构和海量参数量（如百亿级甚至千亿级），在工业、科研和日常应用中发挥着越来越重要的作用。
```

而如果要禁用思考模式，则需要将enable_thinking=True改为False即可。

通过`enable_thinking`参数决定是否启用思考模式，如何设置为`True`，模型回复结果中就会包含思维链的全部内容。此时模型显存占用约62G。

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 530.30.02              Driver Version: 530.30.02    CUDA Version: 12.1 |
|-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf           Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA GeForce RTX 3090         On | 00000000:18:00.0 Off |                  N/A |
| 30%   43C    P2             126W / 350W|  12875MiB / 24576MiB |     19%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  NVIDIA GeForce RTX 3090         On | 00000000:3B:00.0 Off |                  N/A |
| 32%   45C    P2             149W / 350W|  16204MiB / 24576MiB |     31%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  NVIDIA GeForce RTX 3090         On | 00000000:86:00.0 Off |                  N/A |
| 30%   48C    P2             172W / 350W|  16204MiB / 24576MiB |     31%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  NVIDIA GeForce RTX 3090         On | 00000000:AF:00.0 Off |                  N/A |
| 34%   46C    P2             145W / 350W|  15384MiB / 24576MiB |     29%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
```

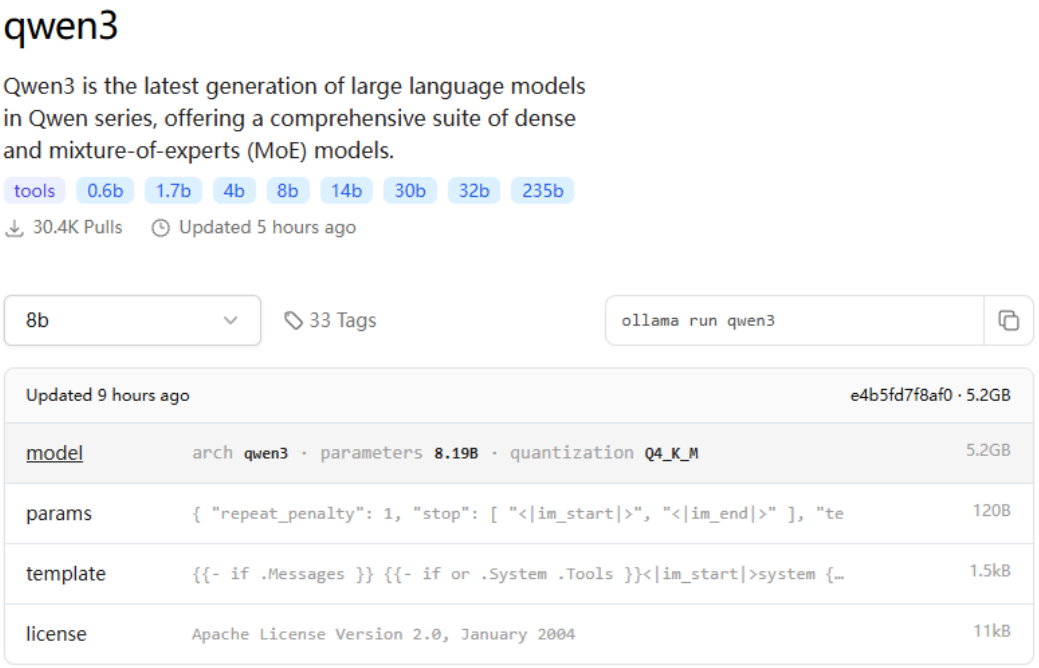
而除了使用原生的`Transformers`库加载以外，目前主流的大模型部署框架以`llama.cpp`、`Ollama`和`vllm`为主，其各自项目在`Github`上的`Stars`增长曲线如下所示。其中`Ollama`的`Stars`增长曲线是断档式领先，而`llama.cpp`和`vllm`的`Stars`增长曲线则相对平缓，处于一个稳步增长的趋势。



三、Qwen3-14B模型接入Ollama与推理流程

`Ollama`是在`Github`上的一个开源项目，其项目定位是：**一个本地运行大模型的集成框架**，目前主要针对主流的`LLaMA`架构的开源大模型设计，通过将模型权重、配置文件和必要数据封装进由`Modelfile`定义的包中，从而实现大模型的下载、启动和本地运行的自动化部署及推理流程。此外，`Ollama`内置了一系列针对大模型运行和推理的优化策略，目前作为一个非常热门的大模型托管平台，基本主流的大模型应用开发框架如`LangChain`、`AutoGen`、`Microsoft GraphRAG`及热门项目`AnythingLLM`、`OpenWebUI`等高度集成。

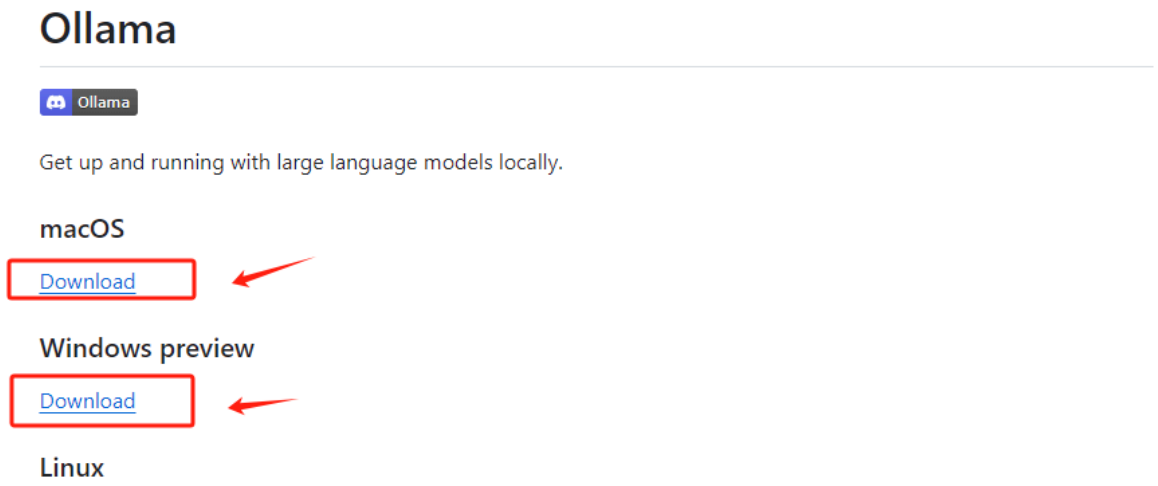
目前Qwen3系列模型已支持使用ollama进行调用，可以在模型主页查看调用情况：https://ollama.com/library/qwen3



1. Ollama项目安装

`Ollama`项目支持跨平台部署，目前已兼容**Mac、Linux和Windows**操作系统。特别地对`Mac`和`Windows`用户提供了非常直观的预览版，包括了内置的`GPU`加速功能、访问完整模型库的能力，以及对`OpenAI`的兼容性在内的`Ollama REST API`，对用户使用尤为友好。

但无论使用哪个操作系统，`Ollama`项目的安装过程都设计得非常简单。根据后续的课程的研发需求以及真实企业的应用需求，我们建议大家使用`Linux`系统进行实践。同时课程也将选择以`Linux`版本为例进行详细介绍。对于其他操作系统版本的安装，大家可以通过如下链接，根据自己的实际情况进行安装体验：https://github.com/ollama/ollama

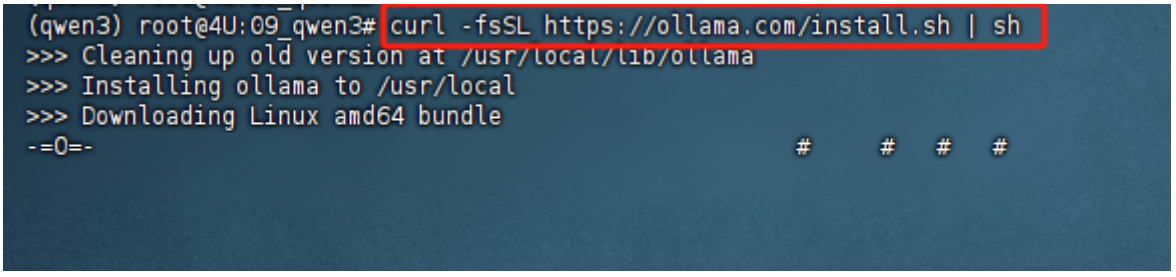


【安装方案一】Ollama在线安装

`Ollama`项目本地安装的方法极为简单，这里我们以`Linux`系统为例，先进入命令行终端，执行如下一条命令行即可自动化完成：

```
curl -fsSL https://ollama.com/install.sh | sh
```

> 注意：`Qwen3`系列模型虽然已经集成Qllama，但运行此模型需要 Ollama 0.6.6 或更高版本。



这行命令的目的是从`https://ollama.com/`网站读取`install.sh`脚本，并立即通过`sh`执行该脚本，在安装过程中会包含以下几个主要的操作：

1. 检查当前服务器的基础环境，如系统版本等；
2. 下载Ollama的二进制文件；
3. 配置系统服务，包括创建用户和用户组，添加Ollama的配置信息；
4. 启动Ollama服务；

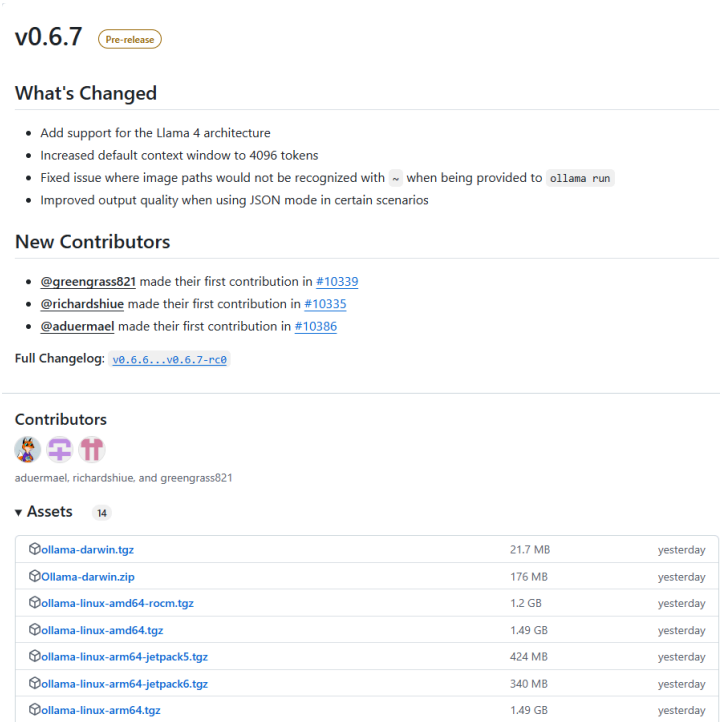
这个过程会比较慢，拉取的文件约2G左右，如果安装过程中未出现任何错误信息，通常情况下能够表明安装已成功。可以通过执行以下命令来检查Ollama服务的运行状态：

```
ollama list
```

```
(kt) root@autodl-container-b4de43b42c-6d000661:~/autodl-tmp/Qwen3/ktransformers# ollama list
NAME                            ID              SIZE      MODIFIED
qwen3:30b-a3b-fp16              5cb412b1ed11    61 GB     13 hours ago
DeepSeek-R1-UD-IQ1_M:latest     98305ae31736    140 GB    4 days ago
gemma3:27b                      a418f5838eaf    17 GB     2 weeks ago
unsloth_model:latest            616b73b8b267    19 GB     6 weeks ago
qwq-32b-bnb:latest              530fb8ad66c8    19 GB     6 weeks ago
qwq:latest                      cc1091b0e276    19 GB     7 weeks ago
qwq-32b:latest                  31a7a6f74a79    19 GB     7 weeks ago
deepseek-r1:671b                739e1b229ad7    404 GB    2 months ago
nomic-embed-text:latest         0a109f422b47    274 MB    2 months ago
```

【安装方案二】Ollama离线安装

更为一般的情况下，推荐使用`Ollama`离线部署。我们可以在`Ollama Github`主页查看目前`Ollama`支持的各操作系统安装包：https://github.com/ollama/ollama/releases



v0.6.7 Pre-release

What's Changed

- Add support for the Llama 4 architecture
- Increased default context window to 4096 tokens
- Fixed issue where image paths would not be recognized with `~` when being provided to `ollama run`
- Improved output quality when using JSON mode in certain scenarios

New Contributors

- @greengrass821 made their first contribution in #10339
- @richardshiue made their first contribution in #10335
- @aduermael made their first contribution in #10386

Full Changelog: v0.6.6...v0.6.7-rc0

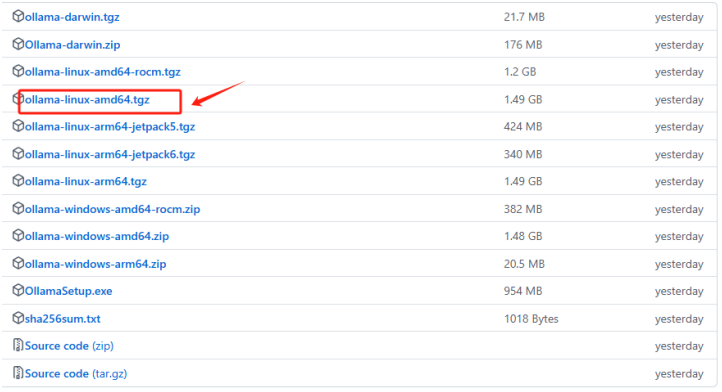
Contributors

aduermael, richardshiue, and greengrass821

▾ Assets 14

ollama-darwin.tgz	21.7 MB	yesterday
Ollama-darwin.zip	176 MB	yesterday
ollama-linux-amd64-rocm.tgz	1.2 GB	yesterday
ollama-linux-amd64.tgz	1.49 GB	yesterday
ollama-linux-arm64-jetpack5.tgz	424 MB	yesterday
ollama-linux-arm64-jetpack6.tgz	340 MB	yesterday
ollama-linux-arm64.tgz	1.49 GB	yesterday

若是`Ubuntu`操作系统，选择其中`ollama-linux-amd64.tgz`下载和安装即可。



ollama-darwin.tgz	21.7 MB	yesterday
Ollama-darwin.zip	176 MB	yesterday
ollama-linux-amd64-rocm.tgz	1.2 GB	yesterday
ollama-linux-amd64.tgz	1.49 GB	yesterday
ollama-linux-arm64-jetpack5.tgz	424 MB	yesterday
ollama-linux-arm64-jetpack6.tgz	340 MB	yesterday
ollama-linux-arm64.tgz	1.49 GB	yesterday
ollama-windows-amd64-rocm.zip	382 MB	yesterday
ollama-windows-amd64.zip	1.48 GB	yesterday
ollama-windows-arm64.zip	20.5 MB	yesterday
OllamaSetup.exe	954 MB	yesterday
sha256sum.txt	1018 Bytes	yesterday
Source code (zip)		yesterday
Source code (tar.gz)		yesterday

此外，`Ollama`的离线版本安装包也可从网盘中下载：



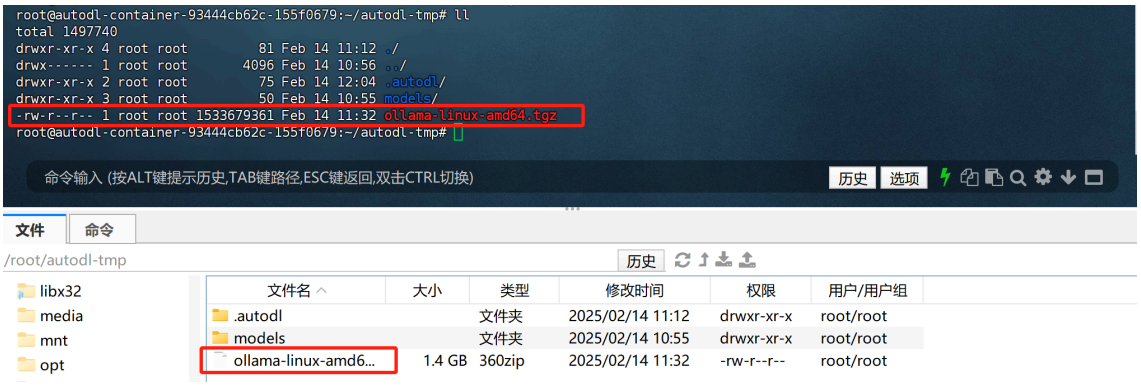
我的网盘 > Qwen3模型本地部署与调用 > 配套工具

全部文件

文件名	大小	类型
ollama-linux-amd64.tgz	1.49GB	tgz文件

大家扫码即可下载。

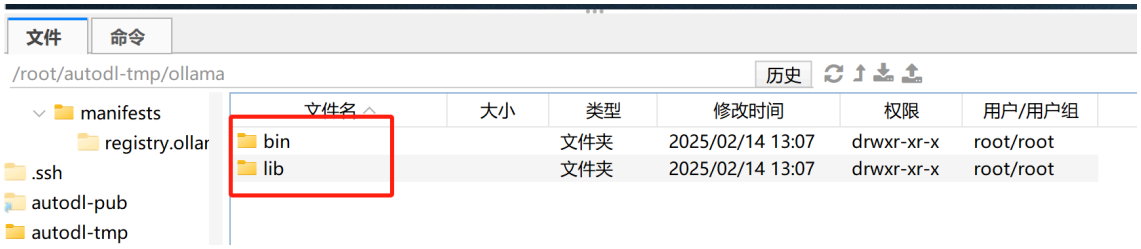
- 下载完成后，需要先上传至服务器：



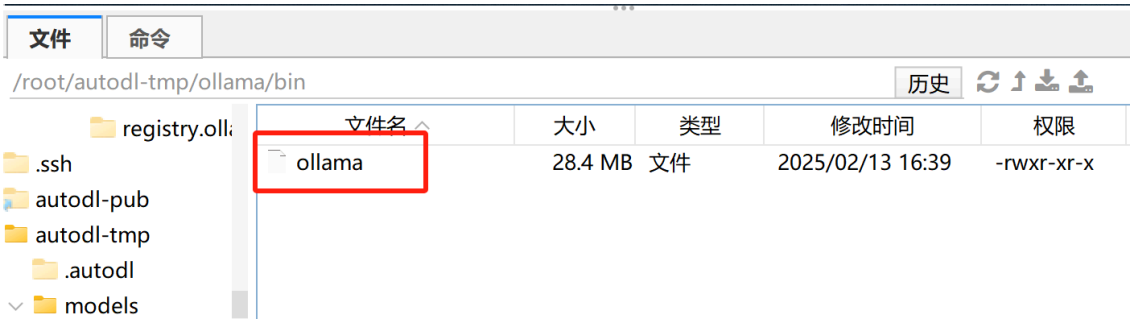
然后使用如下命令进行解压缩

```
mkdir ./ollama
tar -zxvf ollama-linux-amd64.tgz -C ./ollama
```

解压缩后项目文件如图所示：



而在bin中，可以找到ollama命令的可执行文件。



此时，我们可以使用如下方式使用ollama：

```
cd ./bin
./ollama help
```

```
root@autodl-container-eb2e4cadca-dbe44660:~/autodl-tmp/ollama/bin# ./ollama help
Large language model runner

Usage:
  ollama [flags]
  ollama [command]

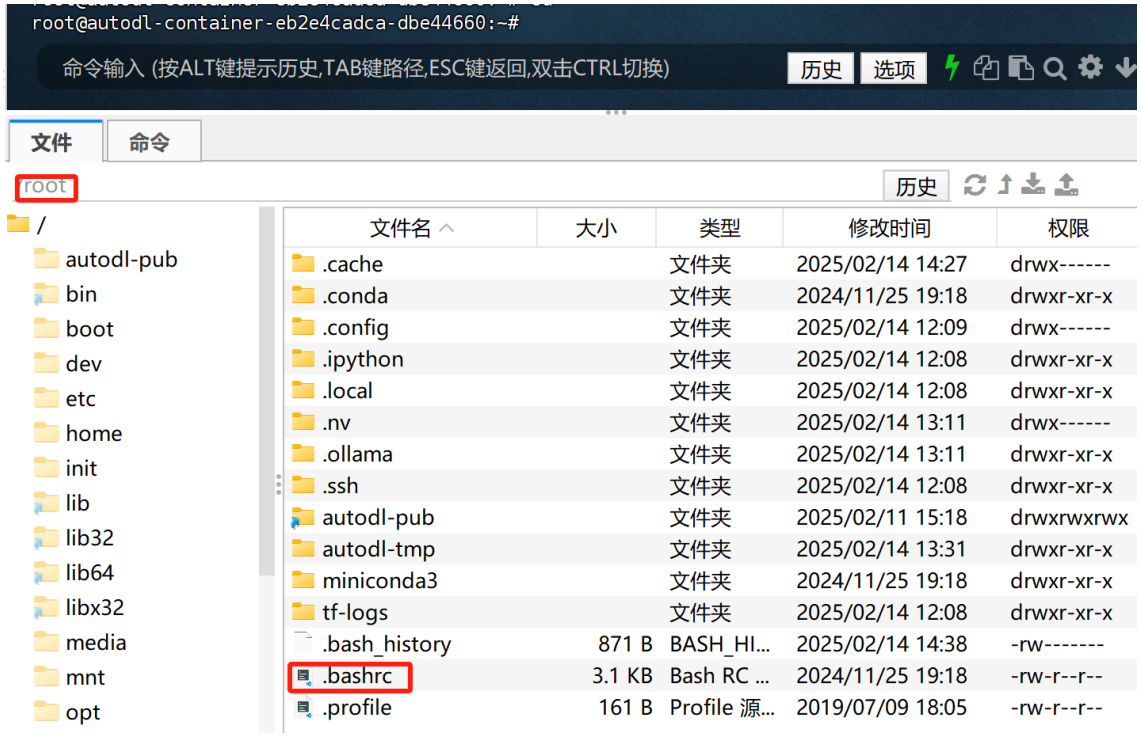
Available Commands:
  serve       Start ollama
  create      Create a model from a Modelfile
  show        Show information for a model
  run         Run a model
  stop        Stop a running model
  pull        Pull a model from a registry
  push        Push a model to a registry
  list        List models
  ps          List running models
  cp          Copy a model
  rm          Remove a model
  help        Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version   Show version information

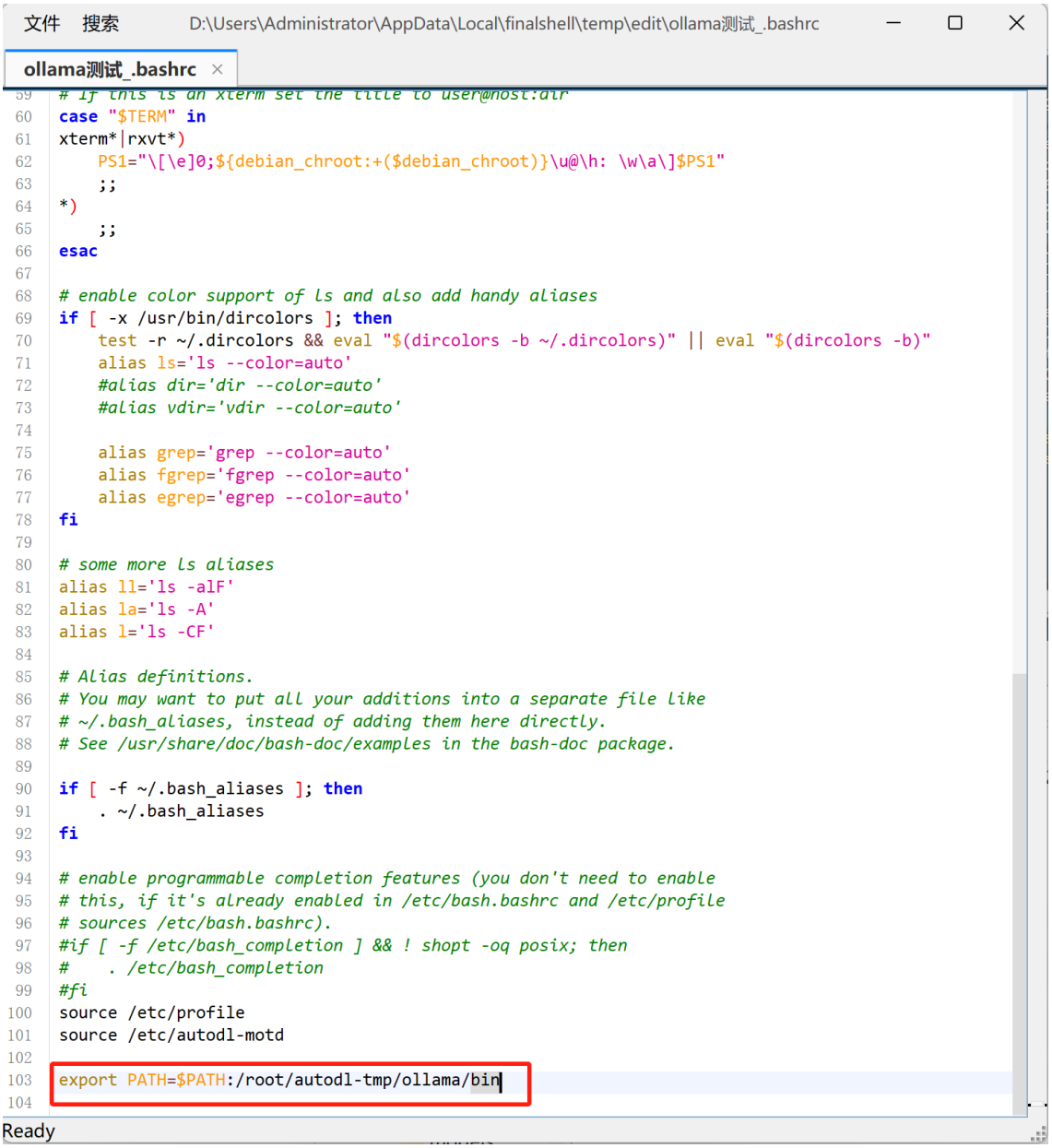
Use "ollama [command] --help" for more information about a command.
```

> 此处若显示没有可执行权限，可以使用如下命令为当前脚本添加可执行权限：
>
> ```
> chmod +x ollama
> ```

而为了使用命令方便，我们也可以将脚本文件写入环境变量中。我们可以在主目录（root）下找到.bashrc文件：



然后在`.bashrc`文件结尾写入ollama/bin文件路径：




```
export PATH=$PATH:/root/autodl-tmp/ollama/bin
```

保存并退出后，输入如下命令来使环境变量生效：

```
source ~/.bashrc
```

然后在任意路径下输入如下命令，测试ollama环境变量是否生效

```
ollama help
```

```
root@autodl-container-eb2e4cadca-dbe44660:~# source ~/.bashrc
root@autodl-container-eb2e4cadca-dbe44660:~# ollama help
Large language model runner

Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve       Start ollama
  create      Create a model from a Modelfile
  show        Show information for a model
  run         Run a model
  stop        Stop a running model
  pull        Pull a model from a registry
  push        Push a model to a registry
  list        List models
  ps          List running models
  cp          Copy a model
  rm          Remove a model
  help        Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version   Show version information

Use "ollama [command] --help" for more information about a command.
```

- 【可选】更换Ollama默认模型权重下载地址

  接下来我们需要使用ollama来下载模型，但默认情况下，ollama会将模型下载到/root/.ollama文件夹中，会占用系统盘空间，因此，若有需要，可以按照如下方法更换模型权重下载地址。

  此外无论是在线还是离线安装的ollama，都可以按照如下方法更换模型权重下载地址。还是需要打开`/root/.bashrc`文件，写入如下代码：

  ```
  export OLLAMA_MODELS=/root/autodl-tmp/models
  ```

```
    ;;
*)
    ;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#    . /etc/bash_completion
#fi
source /etc/profile
# source /etc/autodl-motd

export PATH=$PATH:/root/autodl-tmp/ollama/bin
export OLLAMA_MODELS=/root/autodl-tmp/models
```

> 这里的路径需要改写为自己的文件地址

保存并退出后，输入如下命令来使环境变量生效：

```
source ~/.bashrc
```

测试环境变量是否生效

```
echo $OLLAMA_MODELS
```

```
root@autodl-container-eb2e4cadca-dbe44660:~# echo $OLLAMA_MODELS
/root/autodl-tmp/models
root@autodl-container-eb2e4cadca-dbe44660:~#
```

- 启动ollama

  接下来即可启动ollama，为后续下载模型做准备：

```
ollama start
```

```
root@autodl-container-eb2e4cadca-dbe44660:~# ollama start
2025/02/14 15:05:04 routes.go:1186: INFO server config env="map[CUDA_VISIBLE_DEVICES: GPU_DEVICE_ORDINAL
: HIP_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_DEBUG:false O
LLAMA_FLASH_ATTENTION:false OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://127.0.0.1:11434 OLLAMA_INTEL_GPU:fa
lse OLLAMA_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBRARY: OLLAMA_LOAD_TIMEOUT:5m0s OLLAMA_MAX
_LOADED_MODELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MODELS:/root/autodl-tpm/models OLLAMA_MULTIUSER_CACHE:false
 OLLAMA_NOHISTORY:false OLLAMA_NOPRUNE:false OLLAMA_NUM_PARALLEL:0 OLLAMA_ORIGINS:[http://localhost http
s://localhost http://localhost:* https://localhost:* http://127.0.0.1 https://127.0.0.1 http://127.0.0.1
:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file://
* tauri://* vscode-webview://*] OLLAMA_SCHED_SPREAD:false ROCR_VISIBLE_DEVICES: http_proxy: https_proxy:
 no_proxy:]"
time=2025-02-14T15:05:04.916+08:00 level=INFO source=images.go:432 msg="total blobs: 0"
time=2025-02-14T15:05:04.916+08:00 level=INFO source=images.go:439 msg="total unused blobs removed: 0"
time=2025-02-14T15:05:04.917+08:00 level=INFO source=routes.go:1237 msg="Listening on 127.0.0.1:11434 (v
ersion 0.5.9)"
time=2025-02-14T15:05:04.917+08:00 level=INFO source=gpu.go:217 msg="looking for compatible GPUs"
time=2025-02-14T15:05:05.298+08:00 level=INFO source=types.go:130 msg="inference compute" id=GPU-8af1bad
3-ea66-60aa-1913-824cf10463e5 library=cuda variant=v12 compute=8.9 driver=12.6 name="NVIDIA GeForce RTX
4090" total="23.5 GiB" available="23.2 GiB"
```

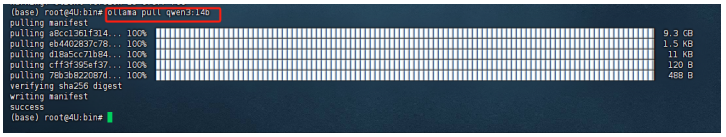
> 注意，在整个应用使用期间，需要持续开启Ollama。

2. Qwen3-14B GGUF格式模型权重下载

ollama模型权重下载可以使用ollama pull命令进行在线下载（没有网络门槛，且速度很快），当然也可以手动下载模型权重，然后上传到服务器上，再注册到Ollama中，就可以通过Ollama调用模型了。以下两种模型权重下载方法任选其一执行即可。

2.1 【方案一】在线下载模型权重

ollama中使用pull命令即可直接下载模型权重。



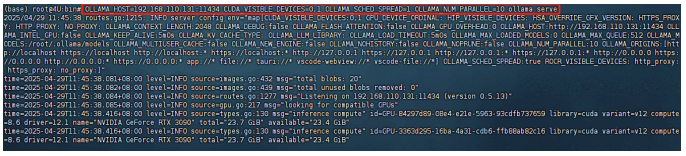

```
ollama run qwen3:14B
```

> 注，在使用pull命令下载模型前，需要先启动ollama。

使用该命令后系统就会自动下载模型权重，并自动完成注册。等待模型下载完成后即可直接使用。

> 除此以外，也可以通过`ollama serve`更灵活的指定启动参数。这里有一些关键的参数，可以控制`ollama`服务的启动行为：
>
> - **OLLAMA_HOST**：设置`ollama`服务的监听地址，默认是`127.0.0.1:11434`，如果需要指定其他地址，可以设置为`公网IP:11434`；
> - **CUDA_VISIBLE_DEVICES**：设置`ollama`服务使用的`GPU`设备，默认选择所有可用的`GPU`设备，如果需要指定其他`GPU`设备，可以设置为`1`，`2`等，用逗号分隔；
> - **OLLAMA_SCHED_SPREAD**：设置`ollama`所选择`GPU`资源是否均匀分布，默认是`true`，如果设置为`false`，则`ollama`会优先选择性能最好的`GPU`设备；
> - **OLLAMA_NUM_PARALLEL**：设置`ollama`每个模型可以同时处理的最大并行请求数量。默认值会根据可用显（内）存自动选择`4`或`1`。
> - **OLLAMA_MAX_QUEUE**：如果在已经加载一个或多个模型的同时，没有足够的可用内存来加载新的模型请求，则所有新请求将排队，直到可以加载新型号为止。随着先前的模型闲置，将卸载一个或多个，以腾出空间为新型号腾出空间。排队的请求将按顺序处理；
> - **OLLAMA_MAX_LOADED_MODELS**：设置最大加载的模型数量，默认是 3 * GPU 的数量，如果超过这个数量，新的请求会被拒绝；

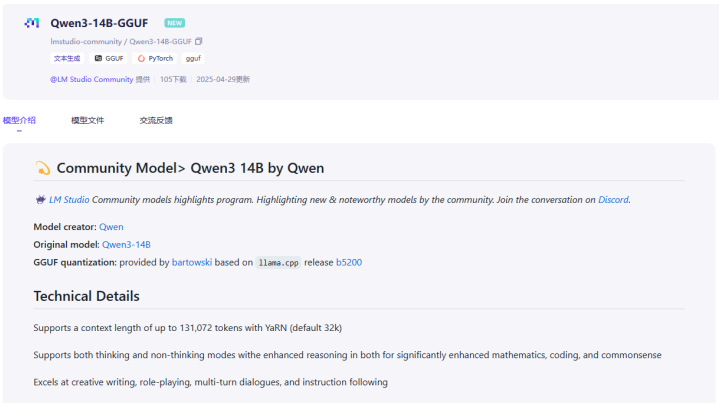
> 大家可以根据自己的需求，灵活设置参数组合，比如：



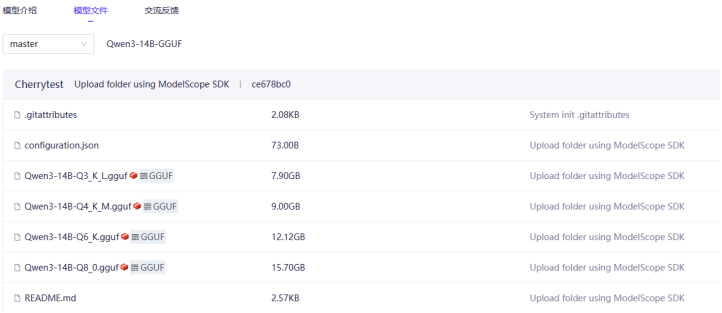
```
OLLAMA_HOST=192.168.110.131:11434 CUDA_VISIBLE_DEVICES=0,1
OLLAMA_SCHED_SPREAD=1 OLLAMA_NUM_PARALLEL=10 ollama serve
```

2.2 【方案二】离线下载模型权重

除了使用ollama run命令外，ollama也支持调用手动下载的自定义模型，但需要是GGUF格式。目前Qwen3模型已有各种不同量化版本的GGUF模型在魔搭社区和huggingface中上线了：https://modelscope.cn/models/lmstudio-community/Qwen3-14B-GGUF



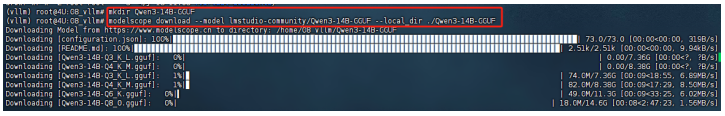
其中每个GGUF量化的权重都包含多个版本：



其中ollama调用的就是Q4_K_M量化版本。这里我们还是考虑下载Q4_K_M量化版本的GGUF权重进行调用，具体执行流程如下。

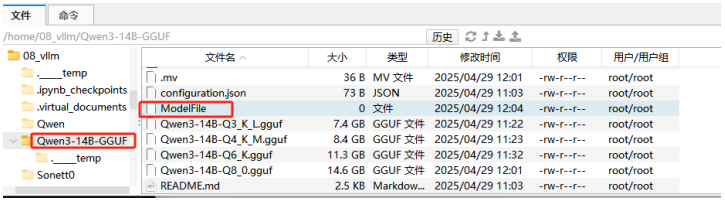
- 下载GGUF格式权重

```
cd ~/autodl-tmp
mkdir Qwen3-14B-GGUF
modelscope download --model lmstudio-community/Qwen3-14B-GGUF --local_dir
./Qwen3-14B-GGUF
```



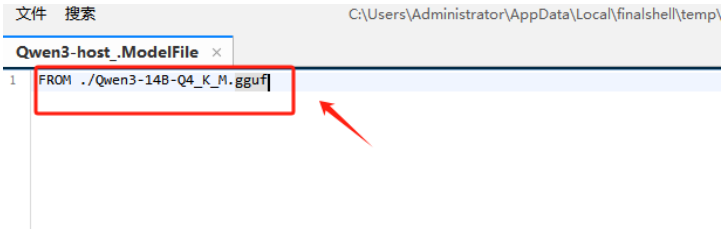
- 注册模型

然后需要创建一个file文件，用于进行ollama模型注册：



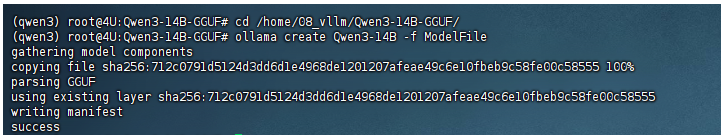
然后在File文件中写入自定义模型GGUF权重地址：

```
FROM ./qwen3-14b-q4_k_m.gguf
```



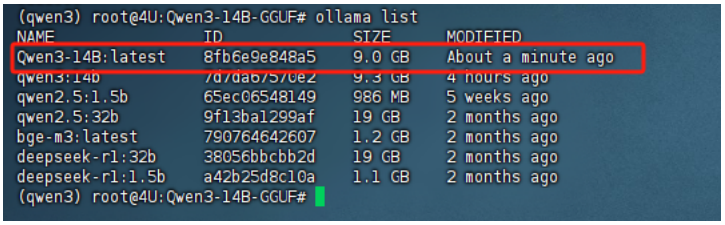
然后，将该模型加入Ollama本地模型列表：

```
cd /home/08_vllm/Qwen3-14B-GGUF/
ollama create Qwen3-14B -f ModelFile
```



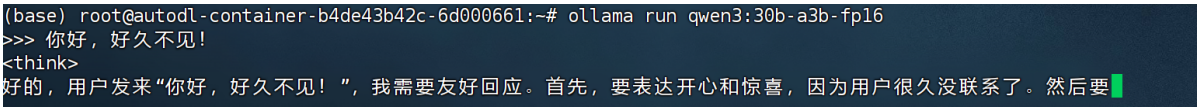
查看模型是否注册成功

```
ollama list
```



接下来即可在命令行中进行调用：

```
ollama run Qwen3-14B
```



后续内容见Jupyter。